Technical Report

Real-Time Data Pipeline Using Apache NiFi, Apache Kafka, and Hadoop HDFS

1. Project Overview

The objective of this project was to design and implement a complete real-time data engineering pipeline capable of simulating, ingesting, transforming, streaming, and storing high-volume streaming data using modern big data technologies.

The entire pipeline was built using:

- Python for real-time streaming data generation
- Apache NiFi for orchestration and stream processing
- Apache Kafka for distributed message streaming
- Hadoop HDFS for distributed storage
- Docker for infrastructure containerization

The project simulates a real-world smart logistics and GPS tracking system where delivery vehicles continuously generate streaming telemetry and shipment tracking data.

The implemented architecture supports:

- Continuous ingestion
- Near real-time processing
- File chunking
- Schema-based validation
- CSV-to-JSON transformation
- Distributed Kafka streaming
- Dynamic HDFS partitioning
- Error handling and retry mechanisms
- Monitoring and queue management

2. High-Level Architecture

```
Python Streaming Generator
        ↓
CSV Streaming Files
        ↓
Apache NiFi
(01_Data_Ingestion)
        ↓
64 KB File Chunking
```

```
(02_Chunking)
        ↓
CSV Validation & JSON Transformation
(03_Transformation)
        ↓
Apache Kafka Topic
(logistics-clean)
        ↓
Apache NiFi Kafka Consumer
        ↓
HDFS Storage
(Date Partitioned)
```

3. Part 1 — Python Streaming Data Generator

3.1 Design Objectives

A Python-based streaming generator was developed to simulate real-time logistics and GPS tracking events.

The generator was intentionally designed to produce both valid and corrupted records in order to test the resilience of the downstream data engineering pipeline.

The generator continuously creates CSV files and writes them into the shared Docker input volume monitored by Apache NiFi.

3.2 Streaming Behavior

The generator operates continuously in a loop to simulate a live production environment.

Each generated file contains streaming shipment events such as:

- Vehicle location
- Driver information
- Shipment status
- GPS coordinates
- Speed readings
- Delivery distance
- Package weight

The generator automatically creates new CSV files at fixed intervals to emulate real-time data arrival.

3.3 Data Quality Simulation

To simulate realistic enterprise streaming environments, the generator intentionally injects multiple types of data anomalies.

Error Type	Description
Missing Values	Empty fields inside optional columns
Duplicate Records	Repeated shipment events
Invalid Records	Text values inside numeric columns
Corrupted Rows	Structurally malformed CSV rows
Numeric Inconsistencies	Negative weights and unrealistic values

Example invalid data:

```
EVT1001,SHIP22,VEH05,DRV02,,15.22,33.12,SPEED_SENSOR_ERROR,-
50.5,120,DELIVERED,2026-05-17 10:20:11
```

This approach helped evaluate:

- Schema validation
- Error routing
- Transformation reliability
- Kafka streaming stability
- HDFS storage behavior

3.4 Engineering Best Practices Applied

Several engineering best practices were implemented during generator development.

Atomic File Spooling

The generator first writes data into temporary `.tmp` files before renaming them into `.csv`.

This prevents Apache NiFi from reading partially written files.

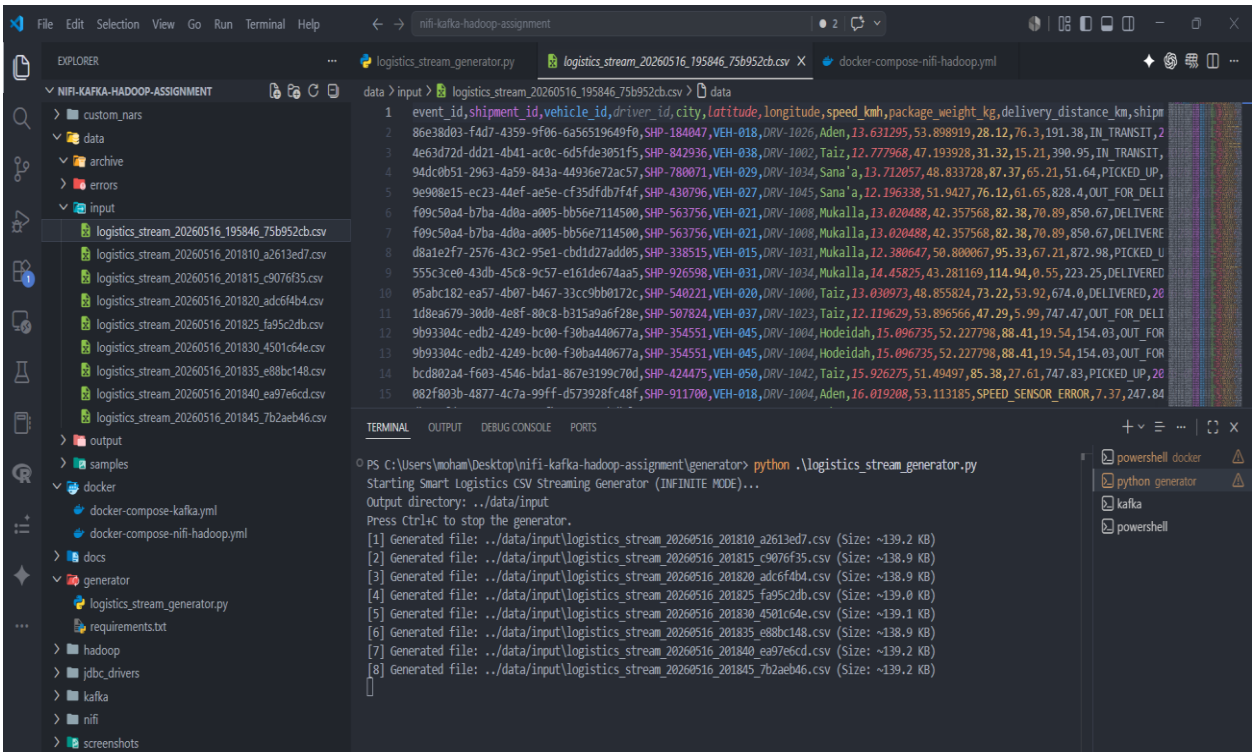
```
.tmp  →  .csv
```

Controlled File Sizing

The generated files were intentionally sized to exceed 64 KB in order to properly test the chunking requirements implemented in NiFi.

Disk Protection

The generator includes safeguards that prevent uncontrolled file generation during long-running tests.



4. Part 2 — Apache NiFi Data Ingestion Pipeline

4.1 Ingestion Architecture

The ingestion layer was implemented using a dedicated NiFi Process Group named:

`01_Data_Ingestion`

The pipeline follows the enterprise-standard architecture:

`ListFile → FetchFile`

instead of using the legacy `GetFile` processor.

This design improves:

- Scalability
- Reliability
- File tracking
- Separation of concerns

4.2 Flow Design

```
ListFile
    ↓
FetchFile
    ↓
UpdateAttribute
    ↓
Output Port
```

ListFile

The `ListFile` processor continuously monitors the shared Docker input directory.

Main configuration:

Property	Value
Input Directory	/opt/data_input
File Filter	.*.csv$
Minimum File Age	10 sec
Listing Strategy	Tracking Timestamps

The minimum file age was added to ensure files are fully written before ingestion begins.

FetchFile

The `FetchFile` processor loads the actual file content into the NiFi flow.

The completion strategy was configured to automatically move processed files into the archive directory.

Archive path:

```
/opt/data_archive
```

This prevents duplicate ingestion and keeps the input directory clean.

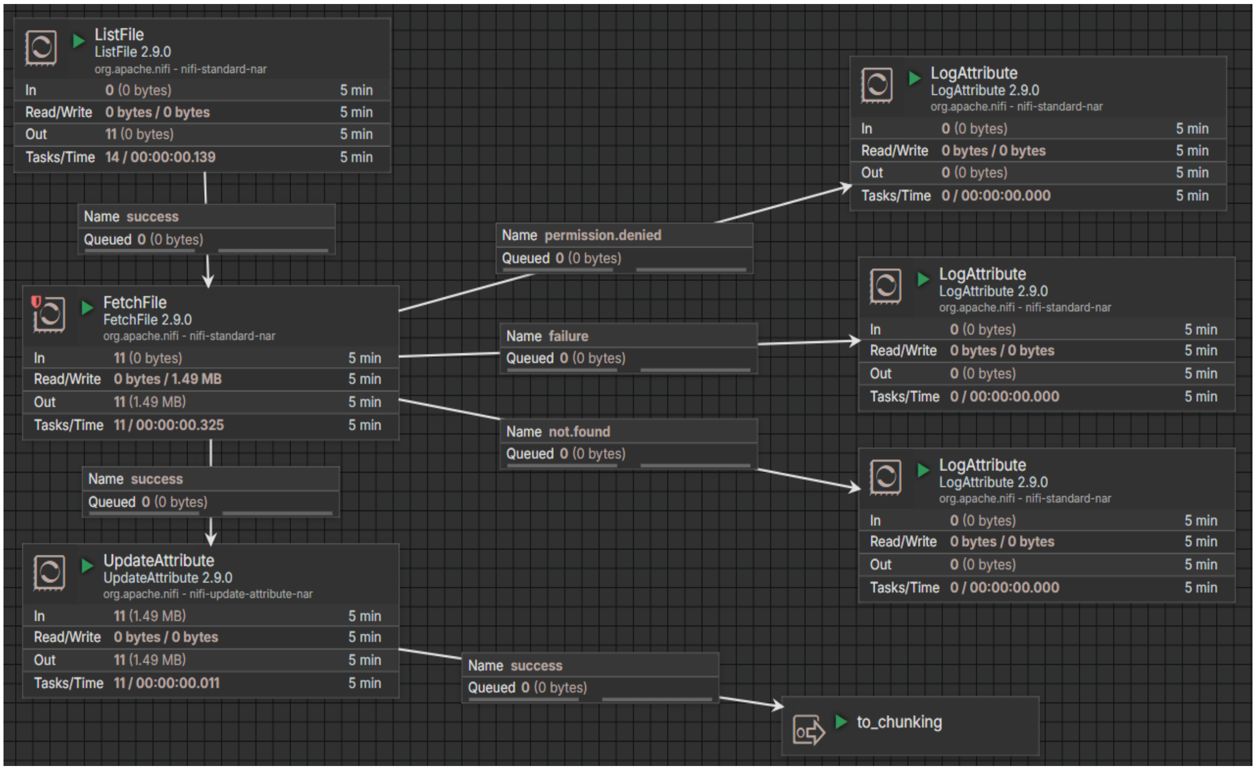
UpdateAttribute

The `UpdateAttribute` processor injects metadata into FlowFiles before routing them to downstream processors.

Example:

```
schema.name = logistics_csv_schema
```

This attribute is later used by the Schema Registry during transformation.



5. Part 3 — File Chunking

5.1 Chunking Objectives

One of the assignment requirements was splitting incoming files into chunks with a maximum size of 64 KB.

This was implemented using the `SplitText` processor.

The chunking stage improves:

- Memory efficiency
- Parallel processing
- Kafka publishing performance
- Stream scalability

5.2 SplitText Configuration

Main configuration:

Property	Value
Maximum Fragment Size	64 KB
Header Line Count	1

5.3 Header Preservation Strategy

The `Header Line Count = 1` configuration was extremely important.

Without header preservation:

- Downstream CSV readers would fail
- Schema validation would break
- JSON conversion would become inconsistent

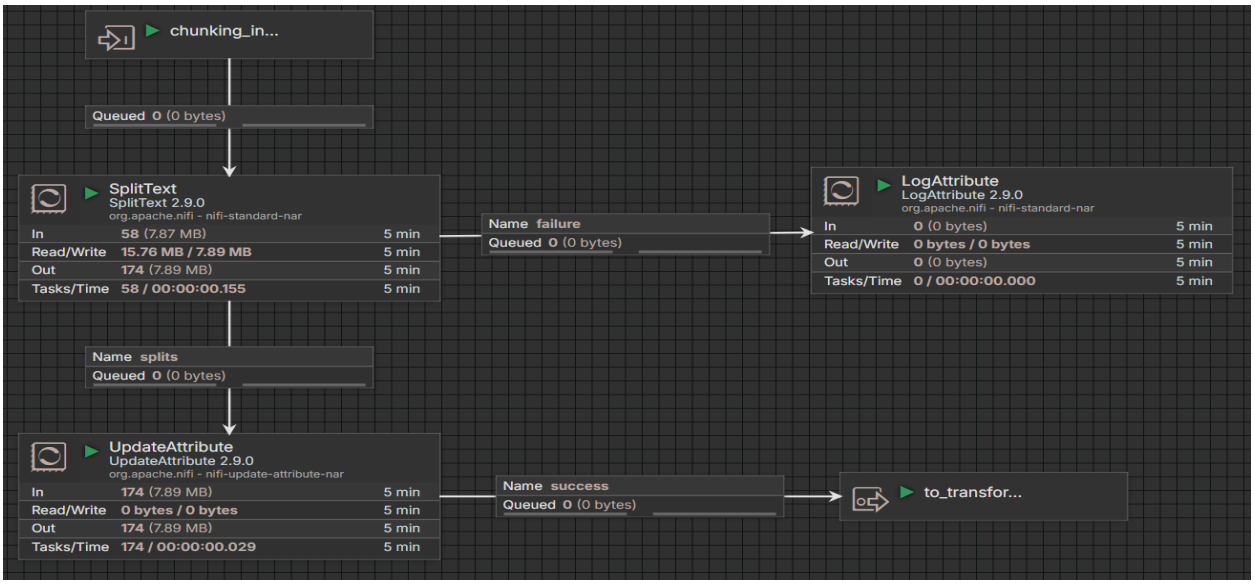
Every generated fragment therefore remains a fully valid CSV file.

5.4 Why Chunking Matters

Large streaming files can create:

- JVM memory pressure
- Slow Kafka publishing
- Queue congestion
- Back Pressure escalation

Splitting the data into smaller fragments significantly improved pipeline stability and streaming throughput.



6. Part 4 — Data Validation & Transformation

6.1 Transformation Objectives

The transformation layer converts streaming CSV records into structured JSON documents suitable for Kafka and HDFS ingestion.

This stage also validates malformed records before they enter the streaming system.

6.2 Schema Registry

An `AvroSchemaRegistry` controller service was created to define the expected structure of incoming data.

The schema included fields such as:

- event_id
- shipment_id
- vehicle_id
- latitude
- longitude
- speed_kmh
- package_weight_kg
- shipment_status
- event_timestamp

This ensured consistent schema enforcement across the entire pipeline.

6.3 CSV to JSON Conversion

The conversion stage was implemented using:

```
CSVReader + JsonRecordSetWriter + ConvertRecord
```

The transformation pipeline:

```
CSV Input
    ↓
CSVReader
    ↓
ConvertRecord
    ↓
JsonRecordSetWriter
    ↓
JSON Output
```

Example JSON output:

```
{
  "event_id": "EVT1021",
  "vehicle_id": "VEH04",
  "speed_kmh": 72.5,
  "shipment_status": "IN_TRANSIT"
}
```

6.4 Invalid Record Handling

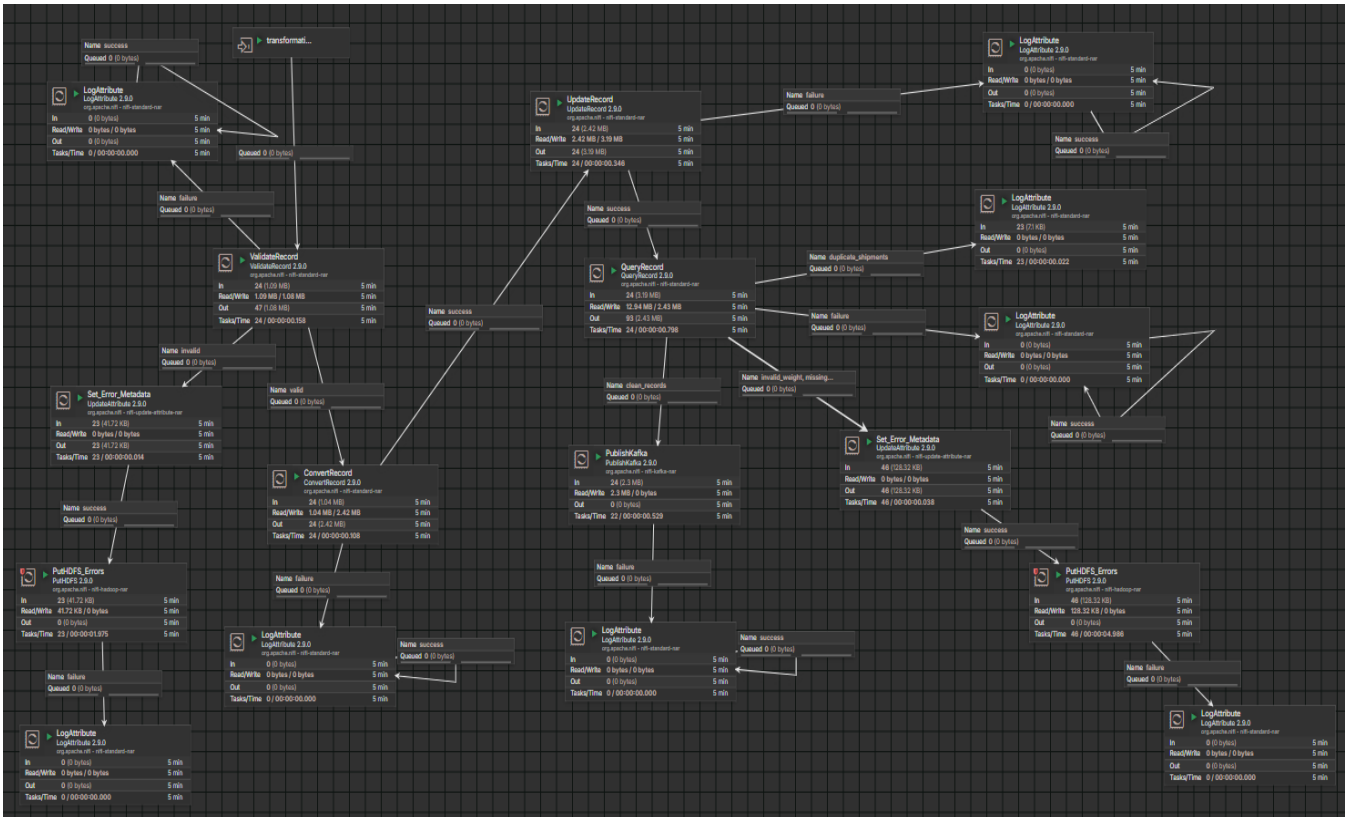
Malformed records are automatically routed into failure relationships.

Examples:

- Invalid numeric values
- Corrupted rows
- Schema mismatch
- Missing required fields

Instead of stopping the entire pipeline, invalid records are isolated and redirected for monitoring and analysis.

This design improves overall pipeline resilience.



7. Part 5 — Apache Kafka Integration

7.1 Kafka Architecture

Apache Kafka was integrated as the central streaming layer between NiFi and Hadoop.

Kafka provides:

- High-throughput streaming
- Decoupled architecture
- Stream buffering
- Replay capability
- Fault tolerance

Topic used:

```
logistics-clean
```

7.2 Topic Configuration

The Kafka topic was created with:

```
kafka-topics.sh --create \
--topic logistics-clean \
--bootstrap-server kafka1:19092 \
--partitions 3 \
--replication-factor 3
```

This configuration improves:

- Parallelism
- Scalability
- High availability

7.3 PublishKafka Configuration

The NiFi processor used:

```
PublishKafka
```

Important configuration:

Property	Value
Topic Name	logistics-clean
Delivery Guarantee	acks=all
Compression Type	none
Transactions Enabled	false

The producer was configured with:

```
acks = all
```

to ensure records are fully replicated before acknowledgment.

7.4 Deterministic Partitioning Strategy

One of the most important architectural decisions was using:

```
Message Key Field = /vehicle_id
```

This guarantees that all records related to the same vehicle are routed into the same Kafka partition.

Benefits:

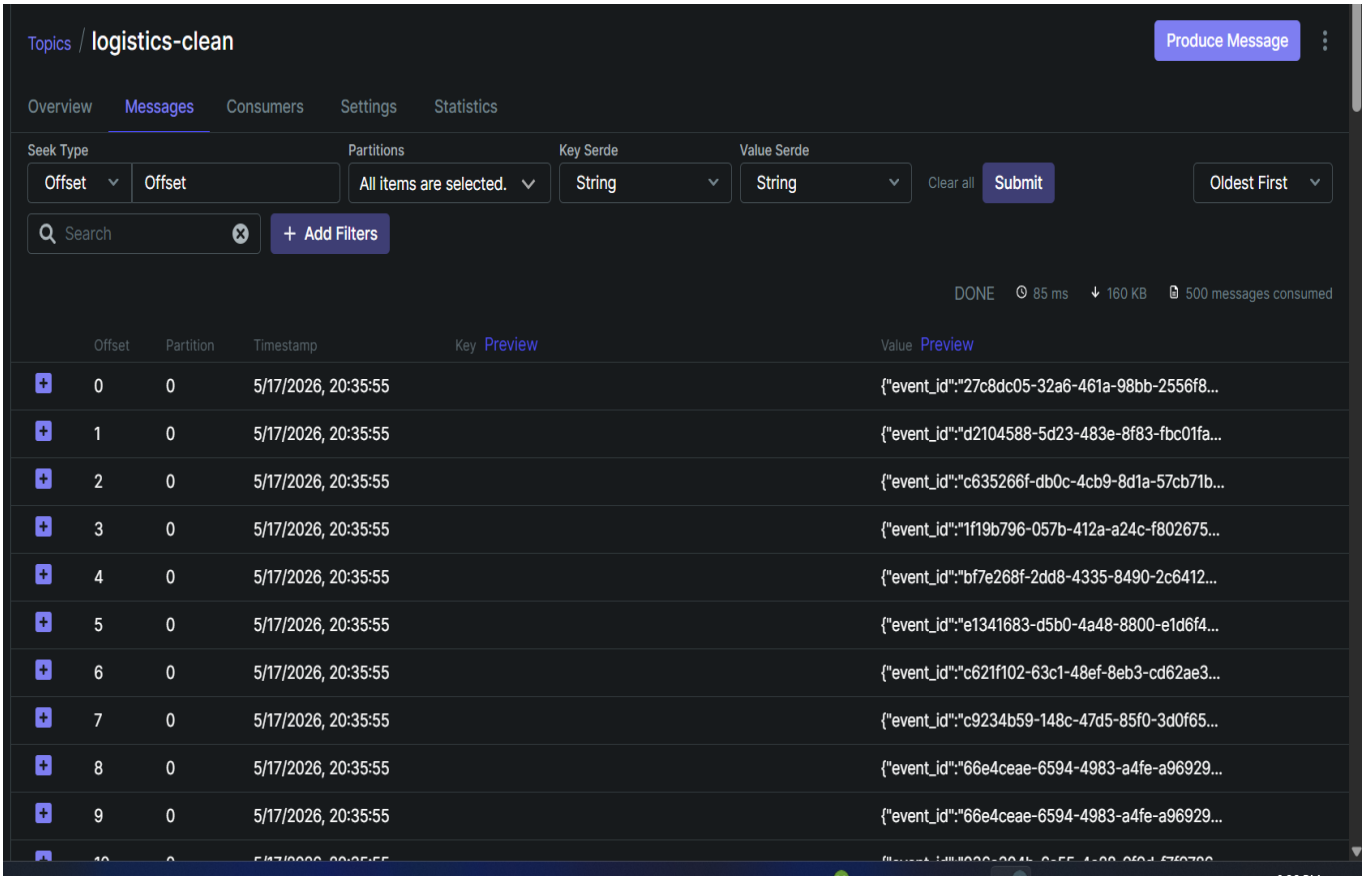
- Preserves event ordering
- Maintains GPS sequence integrity
- Improves downstream analytics consistency

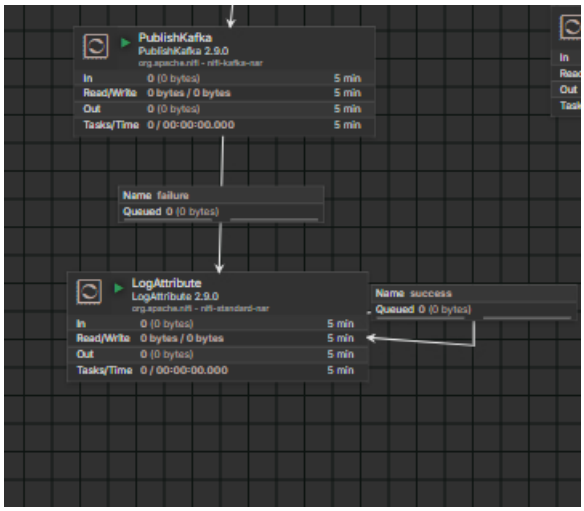
7.5 Kafka Consumer Layer

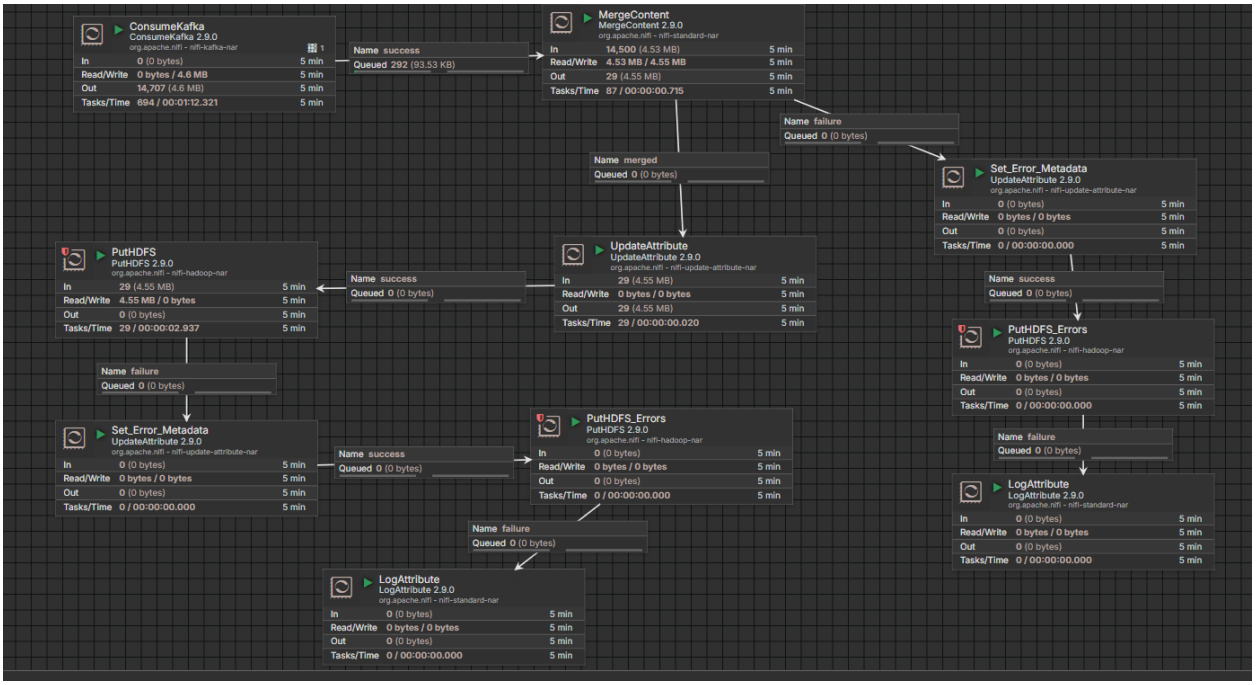
NiFi consumed records using:

```
ConsumeKafkaRecord_2_6
```

The consumer group continuously streamed validated JSON records from Kafka into Hadoop HDFS.


Topics / logistics-clean
Produce Message
Overview
Messages
Consumers
Settings
Statistics
Seek Type
Offset
Offset
Partitions
All items are selected.
Key Serde
String
Value Serde
String
Clear all
Submit
Oldest First
Search
+ Add Filters
DONE
85 ms
160 KB
500 messages consumed
Offset
Partition
Timestamp
Key Preview
Value Preview
0 0 5/17/2026, 20:35:55 {"event_id":"27c8dc05-32a6-461a-98bb-2556f8...
1 0 5/17/2026, 20:35:55 {"event_id":"d2104588-5d23-483e-8f83-fbc01fa...
2 0 5/17/2026, 20:35:55 {"event_id":"c635266f-db0c-4cb9-8d1a-57cb71b...
3 0 5/17/2026, 20:35:55 {"event_id":"1f19b796-057b-412a-a24c-f802675...
4 0 5/17/2026, 20:35:55 {"event_id":"bf7e268f-2dd8-4335-8490-2c6412...
5 0 5/17/2026, 20:35:55 {"event_id":"e1341683-d5b0-4a48-8800-e1d6f4...
6 0 5/17/2026, 20:35:55 {"event_id":"c621f102-63c1-48ef-8eb3-cd62ae3...
7 0 5/17/2026, 20:35:55 {"event_id":"c9234b59-148c-47d5-85f0-3d0f65...
8 0 5/17/2026, 20:35:55 {"event_id":"66e4ceae-6594-4983-a4fe-a96929...
9 0 5/17/2026, 20:35:55 {"event_id":"66e4ceae-6594-4983-a4fe-a96929...


PublishKafka
PublishKafka 2.9.0
In 0 (0 bytes) 5 min
Read/Write 0 bytes / 0 bytes 5 min
Out 0 (0 bytes) 5 min
Tasks/Time 0 / 00:00:00.000 5 min
Name failure
Queued 0 (0 bytes)
LogAttribute
LogAttribute 2.9.0
In 0 (0 bytes) 5 min
Read/Write 0 bytes / 0 bytes 5 min
Out 0 (0 bytes) 5 min
Tasks/Time 0 / 00:00:00.000 5 min
Name success
Queued 0 (0 bytes)



8. Part 6 — Hadoop HDFS Storage

8.1 Storage Objectives

The final stage of the pipeline stores processed JSON records inside Hadoop Distributed File System (HDFS).

This creates the foundation for:

- Data Lake architecture
- Distributed analytics
- Hive integration
- Large-scale batch processing

8.2 PutHDFS Integration

The following NiFi processor was used:

```
PutHDFS
```

The processor was connected to Hadoop using:

- core-site.xml
- hdfs-site.xml

8.3 Dynamic Date Partitioning

The HDFS destination path was dynamically generated using NiFi Expression Language.

Example:

```
/data/year=${now():format('yyyy')}/month=${now():format('MM')}/day=${now():fo
rmat('dd')}/
```

Generated structure:

```
/data/year=2026/month=05/day=17/
```

This partitioning strategy improves:

- Query performance
- Partition pruning
- Storage organization
- Hive compatibility

8.4 Permission Handling

During implementation, permission issues were encountered when NiFi attempted to write into HDFS.

The issue was resolved by:

- Adjusting HDFS ownership
- Updating directory permissions
- Verifying container networking

8.5 Data Distribution Observation

At the end of execution, the generated JSON files appeared as multiple files inside the same daily partition.

This behavior reflects Kafka’s partition-based distribution mechanism and confirms that streaming parallelism was functioning correctly.

Browse Directory

/data/year=2026/month=05/day=17 Go!

Show 25 entries Search:

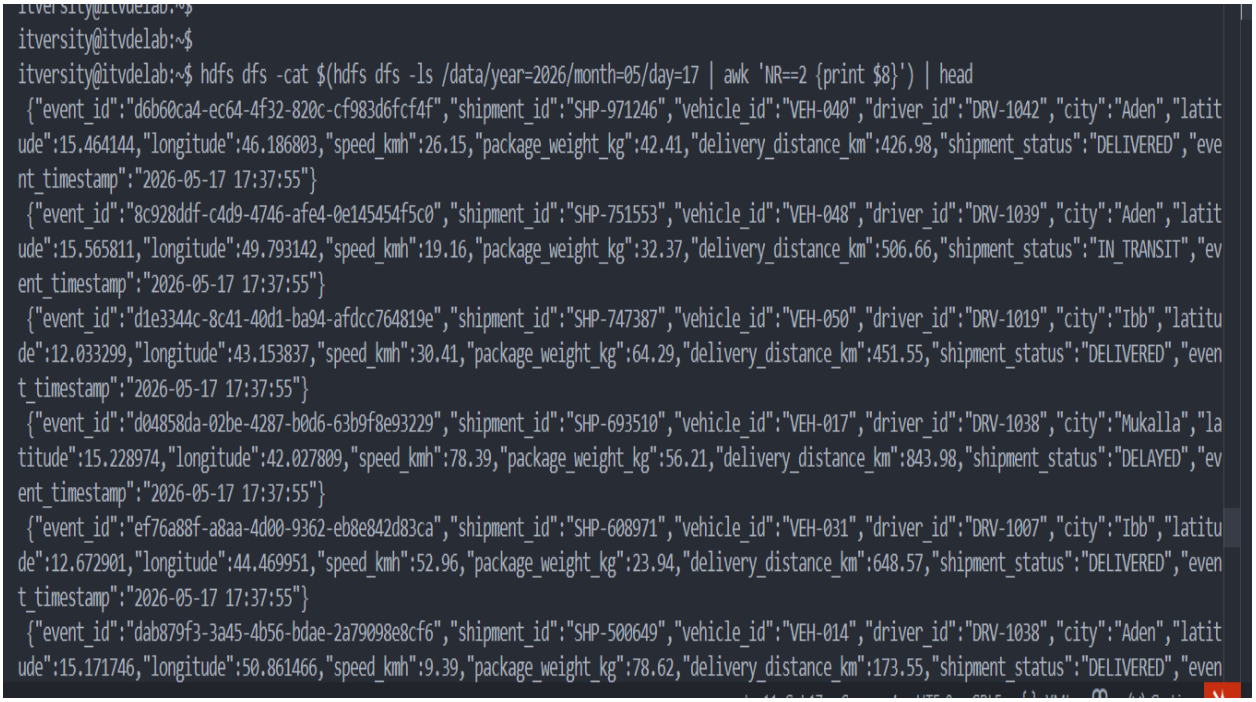
Permission	Owner	Group	Size	Last Modified	Replication	Block Size	Name
-rw-r--r--	nifi	supergroup	160.48 KB	May 17 21:22	3	128 MB	logistics_20260517_182206_0071be83-7ec1-4f6a-906b-056f19296f4d.json
-rw-r--r--	nifi	supergroup	160.57 KB	May 17 21:22	3	128 MB	logistics_20260517_182206_142d9aab-82cf-4ce4-93b2-7d4b0a74e1df.json
-rw-r--r--	nifi	supergroup	160.61 KB	May 17 21:22	3	128 MB	logistics_20260517_182206_16fc2f6c-5884-47bc-94dc-8fb0527786a4.json
-rw-r--r--	nifi	supergroup	160.55 KB	May 17 21:22	3	128 MB	logistics_20260517_182206_a93c81b0-57cb-45f5-9b0c-6d69b65a972e.json
-rw-r--r--	nifi	supergroup	160.49 KB	May 17 21:22	3	128 MB	logistics_20260517_182206_b196e9ae-ef72-4cdb-9f31-ef6ea636e9ab.json
-rw-r--r--	nifi	supergroup	160.65 KB	May 17 21:22	3	128 MB	logistics_20260517_182206_bac01f6e-42dc-4675-8d1a-4053a1e110f8.json
-rw-r--r--	nifi	supergroup	160.54 KB	May 17 21:22	3	128 MB	logistics_20260517_182206_c81fa7f9-28b2-495a-be89-53d7bdccef3d.json

Browse Directory

/errors/stage=transformation-missing-and-invalid/year=2026/month=05/day=17 Go!

Show 25 entries Search:

Permission	Owner	Group	Size	Last Modified	Replication	Block Size	Name
-rw-r--r--	nifi	supergroup	1.89 KB	May 17 22:59	3	128 MB	error__20260517_195910_11633c28-2386-4969-96a6-ff8936e1ecca.json
-rw-r--r--	nifi	supergroup	1.28 KB	May 17 22:59	3	128 MB	error__20260517_195910_27ea7e2f-b3d6-46f4-af17-c7ea4b18ee4f.json
-rw-r--r--	nifi	supergroup	4.49 KB	May 17 22:59	3	128 MB	error__20260517_195910_4d3668d4-f51e-4cf4-bba2-ec176f52eb18.json
-rw-r--r--	nifi	supergroup	6.1 KB	May 17 22:59	3	128 MB	error__20260517_195910_5f84173c-3f89-4d80-8746-69a7e31316e2.json
-rw-r--r--	nifi	supergroup	961 B	May 17 22:59	3	128 MB	error__20260517_195910_a168f179-6513-4c18-8fac-bc08abca8998.json
-rw-r--r--	nifi	supergroup	2.51 KB	May 17 22:59	3	128 MB	error__20260517_195910_db2a3200-4b24-46d0-8e14-4589755a192f.json
-rw-r--r--	nifi	supergroup	328 B	May 17 22:59	3	128 MB	error__20260517_195915_263992ef-e448-43a8-9be7-d9829e2e999e.json



9. Part 7 — Monitoring & Reliability

9.1 Reliability Objectives

The pipeline was designed to remain stable under continuous streaming workloads.

Several reliability mechanisms were implemented to prevent:

- Memory exhaustion
- Queue overflow
- Processor deadlocks
- Data loss

9.2 Back Pressure

Back Pressure was configured on internal queues.

Configuration:

Threshold Type	Value
Object Threshold	10,000
Data Size Threshold	1 GB

This protects the NiFi JVM from OutOfMemory errors during heavy streaming spikes.

9.3 Penalization & Retry Handling

FlowFiles that fail processing are temporarily penalized before retry attempts occur.

Main configuration:

Property	Value
Penalty Duration	30 sec
Yield Duration	1 sec
Retry Count	10

This allows the pipeline to recover automatically from temporary failures without stopping the entire workflow.

9.4 Failed Record Management

Invalid records are routed into dedicated failure paths.

The flow includes:

```
PutHDFS_Errors
```

This processor isolates problematic records for future debugging and analysis.

This approach ensures:

- No silent data loss
- Better troubleshooting
- Pipeline continuity

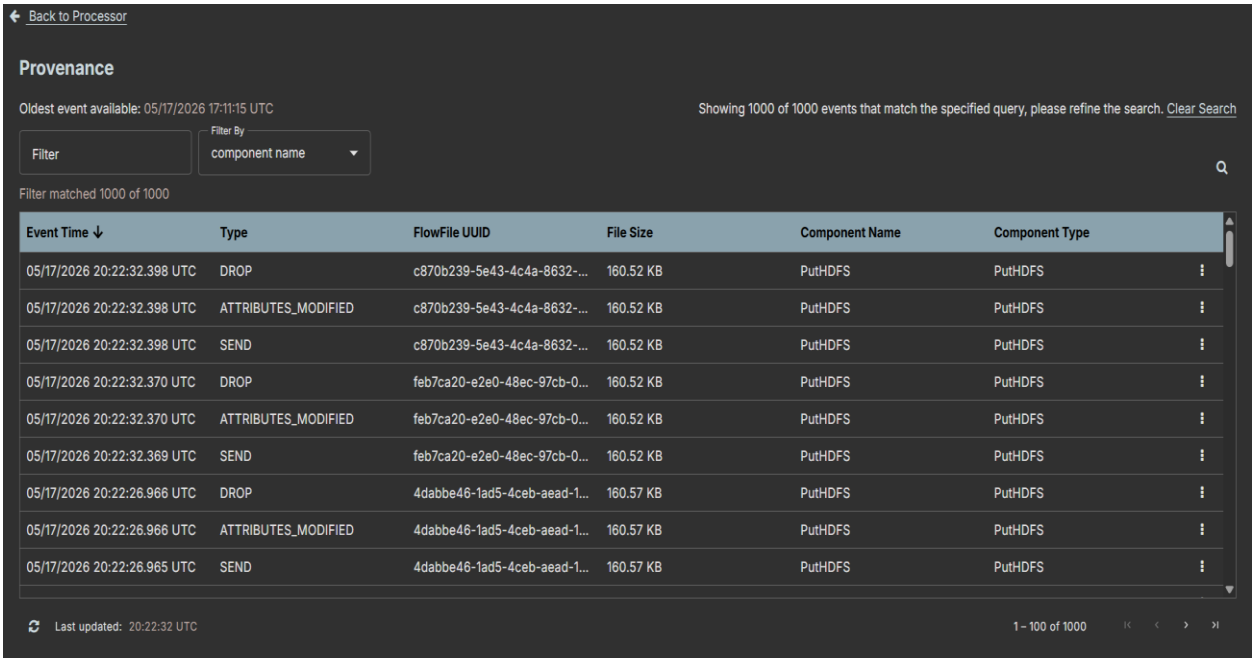
9.5 Provenance Tracking

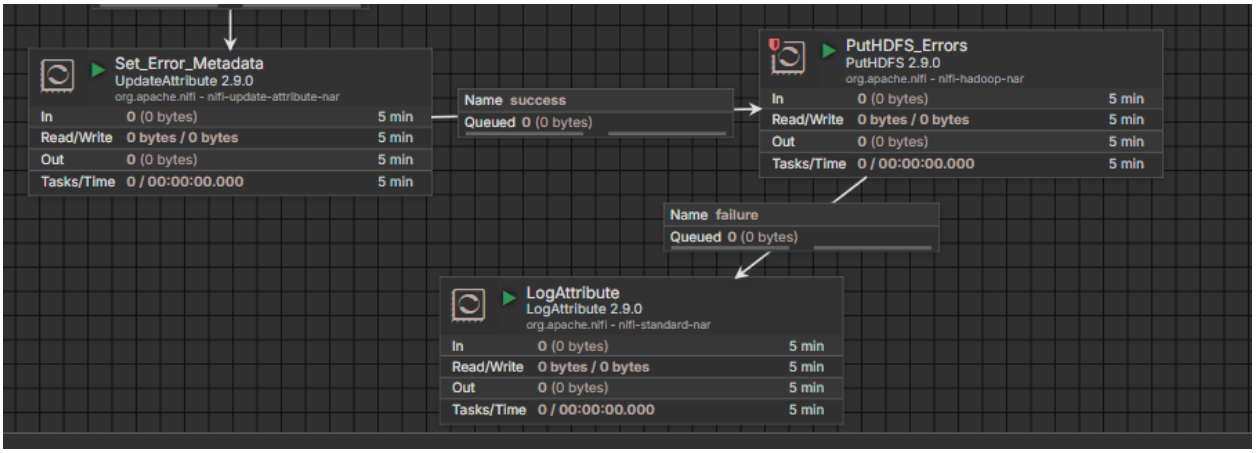
NiFi Provenance was enabled to monitor the lifecycle of each FlowFile.

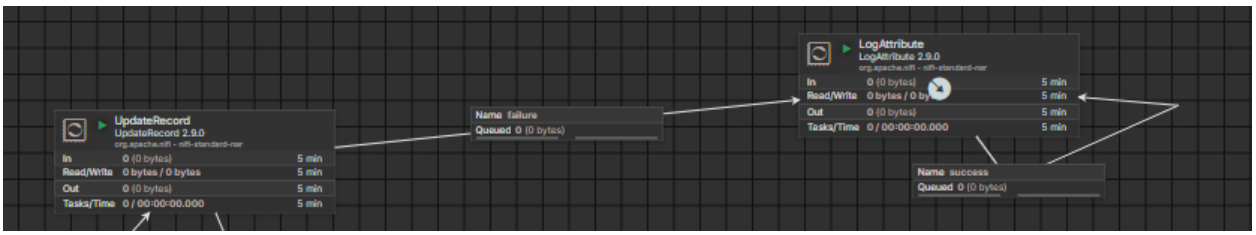
This provided:

- End-to-end traceability

- Debugging visibility
- Queue monitoring
- Performance observation







10. Challenges Faced During Implementation

Several technical challenges were encountered during implementation.

10.1 PutHDFS Processor Availability

Recent NiFi versions no longer include Hadoop processors by default.

Solution:

- Added custom Hadoop NAR libraries
- Mounted external NAR directory into the NiFi container

10.2 HDFS Connection Refused Error

Initially, NiFi failed to connect to HDFS because Hadoop was configured using localhost.

Solution:

- Reconfigured Hadoop networking
- Updated core-site.xml
- Used Docker container hostnames

10.3 HDFS Permission Errors

NiFi encountered `AccessControlException` while writing into HDFS.

Solution:

- Updated HDFS ownership
- Applied proper directory permissions

10.4 Schema Validation Failures

Malformed records caused transformation failures.

Solution:

- Added failure routing
- Implemented retry handling
- Isolated corrupted records

11. Conclusion

This project successfully implemented a complete real-time data engineering pipeline using Apache NiFi, Apache Kafka, and Hadoop HDFS.

The final solution demonstrated:

- Continuous streaming ingestion
- Reliable data orchestration
- Schema-based transformation
- Distributed Kafka streaming
- Dynamic HDFS partitioning
- Error handling and monitoring
- Scalable architecture design

The project also provided practical experience with real-world data engineering challenges including:

- Stream reliability
- Distributed storage
- Queue management
- Fault tolerance
- Data validation
- Infrastructure troubleshooting